

QATIPv3 Azure Lab4

Terraform variables and workspaces

Contents

Lab Objectives	1
Teaching Points.....	1
Before you begin	2
Solution	2
Task1 - Review provisioned terraform files	2
Task2 - Run terraform plan and apply with hardcoded parameter values	3
Task3 - Substitute hardcoded parameter vales with variables	4
Task4 - Override variable values at the command prompt.....	6
Task5 - Utilize terraform.tfvars to supply variable values	7
Task6 - Implement Terraform Workspaces.....	8
Task7 - Lab Clean-up.....	11

Lab Objectives

In this lab you will:

- Deploy resources using a hard-coded terraform file
- Utilize variables to make your code reuseable
- Implement overrides using the command line and tfvars files
- Utilize terraform workspaces to allow multiple simultaneous deployments

Teaching Points

When writing terraform code, it is instinctive to supply explicit values when they are required, for example **name = "michael"**. This can make the code very static though, requiring manual changes of these values each time a modification is needed. A better practice is to use 'placeholders', variables, to which values can be passed at runtime, for example **name = var.username**. Passing values to these variables can be achieved using variable file default values, tfvars files, command line supplied or by being prompted. By default, all changes made to your configuration will be tracked by a single state file. Workspaces allow multiple

state files to exist simultaneously so that the same base code, provided with unique variable values, can be deployed. This makes your code flexible and reusable.

Before you begin

1. Ensure you have completed Lab0 before attempting this lab.
2. In the IDE terminal pane, enter the following command...

```
cd c:\azurelabs\lab4
```

3. This shifts your current working directory to lab4. Ensure all commands are executed in this directory
4. Close any open files and use the Explorer pane to navigate to and open the lab4 folder.

Solution

There is no solution code for this lab as it involves multiple deployment phases. Reach out to your instructor if you encounter issues.

Task1- Review provisioned terraform files

1. Review the provisioned files in c:\azurelabs\lab4..

main.tf Hard coded definition of a resource group named “**RG1**” in location “**northeurope**”. This represents the resources we are to create, in this case just an empty resource group. In production there would be many other resources defined here. There is also an output block (lines 20-22) which displays the current workspace.

variables.tf All content currently commented out

terraform.tfvars All content currently commented out

dev.tfvars All content currently commented out

prod.tfvars All content currently commented out

2. Update line 12 with your lab subscription id and save the change.

Task2- Run terraform plan and apply with hardcoded parameter values

1. Ensure you have navigated to the `c:\azurelabs\lab4` folder
2. Run **terraform init**
3. If required, run **az login** to authenticate to azure and then run **terraform init** again
4. Run **terraform plan**
5. Review the plan output..

```
Terraform will perform the following actions:

# azurerm_resource_group.example will be created
+ resource "azurerm_resource_group" "example" {
  + id          = (known after apply)
  + location    = "northeurope"
  + name        = "RG1"
}

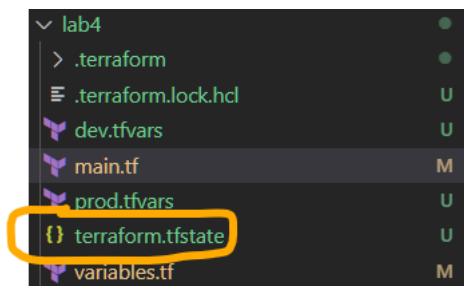
Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ workspace_name = "default"
```

6. Run **terraform apply** followed by **yes**
7. Switch to the Azure portal and verify the creation of resource group RG1 in North Europe..

 **RG1** Azure subscription test North Europe

8. Note the creation of a state file in the root folder..

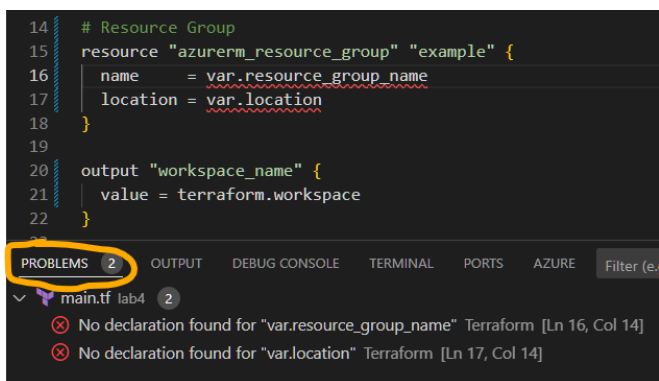


9. **Takeaway:** Hardcoded parameter values will always be used if they exist. This can make the code inflexible and static

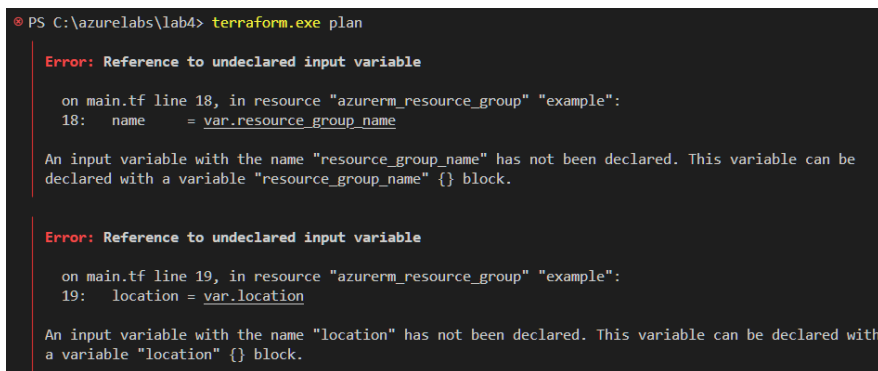
Task3- Substitute hardcoded parameter vales with variables

1. In **main.tf**; Replace hard-code name “RG1” with **var.resource_group_name** (no quotes)
2. Replace hard-coded location “northeurope” with **var.location**
3. Save the changes.

Note that, with terraform extensions installed, the IDE may flag the problem of variables being referenced but not declared. If so, click on the PROBLEMS tab to view...



4. Click back into **TERMINAL**, run **terraform plan** and review the errors...



5. **Takeaway:** If variables are referenced in your code, then they **must** be declared.
6. In **variables.tf**; uncomment all lines except lines 4 and 10 which provide default values for the variables and then save the file..

```

1  variable "resource_group_name" {
2      description = "The name of the Azure Resource Group"
3      type        = string
4      # default    = "RG1"
5  }
6
7  variable "location" {
8      description = "The Azure region to deploy the Resource Group"
9      type        = string
10     # default    = "northeurope"
11 }

```

7. Run **terraform plan**

8. When prompted, enter **northeurope** as the location and **RG1** as the resource group name..

```

PS C:\azurelabs\lab4> terraform.exe plan
var.location
  The Azure region to deploy the Resource Group

  Enter a value: northeurope

var.resource_group_name
  The name of the Azure Resource Group

  Enter a value: RG1

```

9. The planning completes using the values inputted when prompted. Given that these values are the same as the old static values, the planning phase shows that there are no changes needed...

```
No changes. Your infrastructure matches the configuration.
```

10. In **variables.tf**; uncomment lines 4 and 10 and save the file

11. Run **terraform plan**

12. The planning completes using the values drawn from the variables file. Given that these values are the same as the old static values, the planning phase shows that there are no changes needed...

```
No changes. Your infrastructure matches the configuration.
```

13. In **variables.tf**; change the resource group default name (line 4) from **"RG1"** to **"RG2"** and save the file

14. Run **terraform plan**

15. The planning completes using the values drawn from the variables file. Given that the **name** values has changed, terraform flags that the existing resource group would be destroyed, and a new one created if we were to apply this new configuration..

```
Plan: 1 to add, 0 to change, 1 to destroy.
```

16. In **variables.tf**; change the resource group default name (line 4) from “**RG2**” back to “**RG1**” and save the file
17. Destroy the current deployment using **terraform destroy** followed by **yes**
18. **Takeaway:** If a variable is declared but no value has been assigned then you are prompted for values. If a variable is declared with a default value then this value will be used unless overridden. All changes to variable values will apply to the current deployment as you are working in the default workspace. Workspaces allow multiple deployments to exist simultaneously, all using the same code but with different variable values applied as needed. Workspaces will be introduced later in this lab.

Task4- Override variable values at the command prompt

1. Enter the following command..

```
terraform plan -var="location=westus"
```

```
Terraform will perform the following actions:

# azurerm_resource_group.example will be created
+ resource "azurerm_resource_group" "example" {
  + id          = (known after apply)
  + location    = "westus"
  + name        = "RG1"
}

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ workspace_name = "default"
```

2. **Takeaway:** Supplying a variable value at the command prompt using **-var=" "** has the highest priority and overrides values supplied **anywhere** else. In this case **westus** is used as the **location** value as it was entered at the command prompt, whilst the **name** value **RG1** is drawn from the variables file.

Task5- Utilize terraform.tfvars to supply variable values

1. In **terraform.tfvars**; Uncomment line 1 (resource_group_name = "RG2") and save the file

2. Run **terraform plan**

```
Terraform will perform the following actions:

# azure_rm_resource_group.example will be created
+ resource "azure_rm_resource_group" "example" {
  + id          = (known after apply)
  + location    = "northeurope"
  + name        = "RG2"
}

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ workspace_name = "default"
```

3. **Takeaway:** terraform.tfvars values are loaded automatically and take precedence over default values. Therefore the **name** value **RG2** is drawn from terraform.tfvars and the location value **northeurope** is drawn from variables.tf

4. In **terraform.tfvars**; uncomment line 2 (location = "westus") and save the file

5. Run **terraform plan**

```
Terraform will perform the following actions:

# azure_rm_resource_group.example will be created
+ resource "azure_rm_resource_group" "example" {
  + id          = (known after apply)
  + location    = "westus"
  + name        = "RG2"
}

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ workspace_name = "default"
```

6. **Takeaway:** Both the **name** and **location** values are now drawn from terraform.tfvars. When terraform.tfvars supplies **all** variable values, there is no rationale for specifying default values in the variables file as these will always be overridden. It is not uncommon therefore to define the variables using a variables file without declaring any default values.

7. Run **terraform apply** followed by **yes**

8. Switch to the Azure portal and verify the creation of resource group RG2 in West US..

 RG2 Azure subscription test West US

Task6- Implement Terraform Workspaces

1. Create two workspaces; “**dev**” and “**prod**”...

terraform workspace new dev

terraform workspace new prod

terraform workspace list

```
Created and switched to workspace "dev"!

You're now on a new, empty workspace. Workspaces isolate their state,
so if you run "terraform plan" Terraform will not see any existing state
for this configuration.
• PS C:\azurelabs\lab4> terraform workspace new prod
Created and switched to workspace "prod"!

You're now on a new, empty workspace. Workspaces isolate their state,
so if you run "terraform plan" Terraform will not see any existing state
for this configuration.
• PS C:\azurelabs\lab4> terraform workspace list
  default
  dev
  * prod
```

The * indicates your current workspace is **prod**.

The **default** workspace always exists, and this is where your current deployment of resource group RG2 to West US is tracked by the statefile **terraform.tfstate** in the root folder.

2. In EXPLORER, note the creation of a new folder “**terraform.tfstate.d**” with an empty subfolder for each of the new workspaces...

```
▼ terraform.tfstate.d
  ▼ dev
  ▼ prod
```

3. Uncomment all lines in **prod.tfvars** and **dev.tfvars** and save the changes
4. Run **terraform plan --var-file=prod.tfvars**


```
Terraform will perform the following actions:

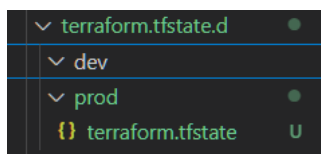
# azurerm_resource_group.example will be created
+ resource "azurerm_resource_group" "example" {
  + id          = (known after apply)
  + location    = "westeurope"
  + name        = "RG3"
}

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ workspace_name = "prod"
```

You are working in the **prod** workspace, and the resource group **name** "RG3" and **location** "westeurope" values are drawn from **prod.tfvars**

5. **Takeaway:** tfvars files other than terraform.tfvars must be applied at the command line using **--var-file=<name of tfvars file>**. Values declared in these files take precedence over terraform.tfvars values if they conflict. Note: If multiple tfvars are applied, then the last loaded has highest priority if there are conflicts. There is no mandatory correlation between the name of the workspace and the name of the tfvars file used in it.
6. Run **terraform apply --var-file=prod.tfvars** and enter **yes**



Note that a new **terraform.tfstate** file is created in the **prod** folder to track the prod workspace deployment

7. Switch to the Azure portal and verify the creation of resource group **RG3** in **West Europe** ..

 RG2	Azure subscription test	West US
 RG3	Azure subscription test	West Europe

Note that RG2 in West US still exists as it is a separate deployment from your default workspace

8. Switch to the dev workspace..

terraform workspace select dev

9. Run `terraform plan --var-file=dev.tfvars`

```
Terraform will perform the following actions:

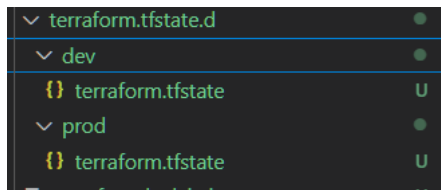
# azurerm_resource_group.example will be created
+ resource "azurerm_resource_group" "example" {
  + id          = (known after apply)
  + location    = "northeurope"
  + name        = "RG4"
}

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ workspace_name = "dev"
```

You are working in the **dev** workspace, and the resource group **name** and **location** values are drawn from **dev.tfvars**

10. Run `terraform apply --var-file=dev.tfvars` followed by **yes**



```
terraform.tfstate.d  ●
dev                  ●
  {} terraform.tfstate  U
  prod
    {} terraform.tfstate  U
```

Note that a new terraform.tfstate file is created in the **dev** folder to track the dev workspace deployment

11. Switch to the Azure portal and verify the creation of resource group **RG4** in **North Europe** ..

 RG2	Azure subscription test	West US
 RG3	Azure subscription test	West Europe
 RG4	Azure subscription test	North Europe

Note that RG2 in West US still exists as it is a separate deployment from your default workspace, as does RG3 in West Europe as it is a separate deployment from your prod workspace

12. **Takeaway:** Workspaces allow the same base code to be used multiple times with varying variable values being applied through the use of named tfvars files. Each workspace creates its own statefile, allowing multiple deployments to exist simultaneously. This is typically used to deploy the same resources across different resource groups/locations.

Task7- Lab Clean-up

1. When deleting resources in workspaces, always pay close attention to workspace you are currently working in. Use **terraform workspace show** to determine your current workspace..

```
PS C:\azurelabs\lab4> terraform workspace show
dev
PS C:\azurelabs\lab4> 
```

2. Destroy the resources in the current workspace (dev) using..
terraform destroy --var-file=dev.tfvars

Note that the tfvars file **must** be specified when performing plan, apply and destroy action

3. Enter **yes** when prompted
4. Switch to the Azure portal to confirm the deletion of the resource group **RG4**
5. The current workspace cannot be deleted. Move to the **prod** workspace and delete the **dev** workspace...

```
terraform workspace select prod
terraform workspace delete dev
```

6. Destroy the resources in the current workspace (prod) using..
terraform destroy --var-file=prod.tfvars
7. Enter **yes** when prompted
8. Switch to the Azure portal to confirm the deletion of the resource group **RG3**

9. The current workspace cannot be deleted. Move to the **default** workspace and delete the **prod** workspace...

terraform workspace select default
terraform workspace delete prod

10. Verify the deletion of the workspaces, leaving just the default workspace. This cannot be deleted..

terraform workspace list

```
PS C:\azurelabs\lab4> terraform.exe workspace list
* default
```

11. Note that the workspace directories are deleted along with the workspaces themselves...

```
> terraform.tfstate.d
```

12. Destroy the resources in the current workspace (default) using..

terraform destroy

13. Enter **yes** when prompted

14. Switch to the Azure portal to confirm the deletion of the resource group **RG2**

Congratulations, you have completed this lab